

Project flow

1. Problem Statement
2. Import libraries
3. Load Dataset
4. Data inspection
5. Data Cleaning
6. Missing Value Analysis
7. EDA
 - 7.1 Univariate analysis
 - 7.2 Bivariate analysis
 - 7.3 Multivariate Analysis
8. Outliers Detection
9. Corr analysis
10. Feature selection
11. Feature engineering -----
12. Encoding Categorical Variable-----
13. Train_test_split
14. Feature scaling
15. Model Building
16. Model prediction
17. Model evaluation
18. Graph check prediction vs actual
19. Error analysis
20. Save model
21. Predict new house price

Problem Statement :

Obj :

Build a Linear Regression Model That predict the selling price of a house using property related features

```
In [1]: # 2 Import all Imp Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#####
from sklearn.model_selection import train_test_split
#####
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
#####
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (mean_absolute_error, mean_squared_error, r2_score)
```

```
import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: # Load Dataset
df=pd.read_csv('train.csv',index_col='Id')
df.head()
```

```
Out[2]:
```

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour
--	------------	----------	-------------	---------	--------	-------	----------	-------------

Id								
1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl
2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl
3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl
4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl
5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl

5 rows × 80 columns



```
In [8]: df.info()
```

<class 'pandas.core.frame.DataFrame'>

Index: 1460 entries, 1 to 1460

Data columns (total 80 columns):

#	Column	Non-Null Count	Dtype
0	MSSubClass	1460 non-null	int64
1	MSZoning	1460 non-null	object
2	LotFrontage	1201 non-null	float64
3	LotArea	1460 non-null	int64
4	Street	1460 non-null	object
5	Alley	91 non-null	object
6	LotShape	1460 non-null	object
7	LandContour	1460 non-null	object
8	Utilities	1460 non-null	object
9	LotConfig	1460 non-null	object
10	LandSlope	1460 non-null	object
11	Neighborhood	1460 non-null	object
12	Condition1	1460 non-null	object
13	Condition2	1460 non-null	object
14	BldgType	1460 non-null	object
15	HouseStyle	1460 non-null	object
16	OverallQual	1460 non-null	int64
17	OverallCond	1460 non-null	int64
18	YearBuilt	1460 non-null	int64
19	YearRemodAdd	1460 non-null	int64
20	RoofStyle	1460 non-null	object
21	RoofMatl	1460 non-null	object
22	Exterior1st	1460 non-null	object
23	Exterior2nd	1460 non-null	object
24	MasVnrType	588 non-null	object
25	MasVnrArea	1452 non-null	float64
26	ExterQual	1460 non-null	object
27	ExterCond	1460 non-null	object
28	Foundation	1460 non-null	object
29	BsmtQual	1423 non-null	object
30	BsmtCond	1423 non-null	object
31	BsmtExposure	1422 non-null	object
32	BsmtFinType1	1423 non-null	object
33	BsmtFinSF1	1460 non-null	int64
34	BsmtFinType2	1422 non-null	object
35	BsmtFinSF2	1460 non-null	int64
36	BsmtUnfSF	1460 non-null	int64
37	TotalBsmtSF	1460 non-null	int64
38	Heating	1460 non-null	object
39	HeatingQC	1460 non-null	object
40	CentralAir	1460 non-null	object
41	Electrical	1459 non-null	object
42	1stFlrSF	1460 non-null	int64
43	2ndFlrSF	1460 non-null	int64
44	LowQualFinSF	1460 non-null	int64
45	GrLivArea	1460 non-null	int64
46	BsmtFullBath	1460 non-null	int64
47	BsmtHalfBath	1460 non-null	int64
48	FullBath	1460 non-null	int64
49	HalfBath	1460 non-null	int64
50	BedroomAbvGr	1460 non-null	int64

```

51 KitchenAbvGr 1460 non-null int64
52 KitchenQual 1460 non-null object
53 TotRmsAbvGrd 1460 non-null int64
54 Functional 1460 non-null object
55 Fireplaces 1460 non-null int64
56 FireplaceQu 770 non-null object
57 GarageType 1379 non-null object
58 GarageYrBlt 1379 non-null float64
59 GarageFinish 1379 non-null object
60 GarageCars 1460 non-null int64
61 GarageArea 1460 non-null int64
62 GarageQual 1379 non-null object
63 GarageCond 1379 non-null object
64 PavedDrive 1460 non-null object
65 WoodDeckSF 1460 non-null int64
66 OpenPorchSF 1460 non-null int64
67 EnclosedPorch 1460 non-null int64
68 3SsnPorch 1460 non-null int64
69 ScreenPorch 1460 non-null int64
70 PoolArea 1460 non-null int64
71 PoolQC 7 non-null object
72 Fence 281 non-null object
73 MiscFeature 54 non-null object
74 MiscVal 1460 non-null int64
75 MoSold 1460 non-null int64
76 YrSold 1460 non-null int64
77 SaleType 1460 non-null object
78 SaleCondition 1460 non-null object
79 SalePrice 1460 non-null int64
dtypes: float64(3), int64(34), object(43)
memory usage: 923.9+ KB

```

```

In [3]: for i in dict(df.isna().sum()).items():
        if i[1]>0:
            print(i[0],i[1])

```

```

LotFrontage 259
Alley 1369
MasVnrType 872
MasVnrArea 8
BsmtQual 37
BsmtCond 37
BsmtExposure 38
BsmtFinType1 37
BsmtFinType2 38
Electrical 1
FireplaceQu 690
GarageType 81
GarageYrBlt 81
GarageFinish 81
GarageQual 81
GarageCond 81
PoolQC 1453
Fence 1179
MiscFeature 1406

```

```
In [8]: # Separating Numerical and categorical Columns
numerical_columns=df.select_dtypes(
    include= ['int64','float64']
).columns
print(f'numerical_columns = {numerical_columns}')
#####
categorical_columns=df.select_dtypes(
    include=['object']
).columns
print(f'Categorical Columns ={categorical_columns}')
```

```
numerical_columns = Index(['MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond',
    'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2',
    'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF',
    'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath',
    'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces',
    'GarageYrBlt', 'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF',
    'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal',
    'MoSold', 'YrSold', 'SalePrice'],
    dtype='object')
Categorical Columns =Index(['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour',
    'Utilities',
    'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2',
    'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st',
    'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation',
    'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2',
    'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual',
    'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual',
    'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature',
    'SaleType', 'SaleCondition'],
    dtype='object')
```

```
In [11]: df.duplicated().sum()
df.drop_duplicates(inplace=True)
```

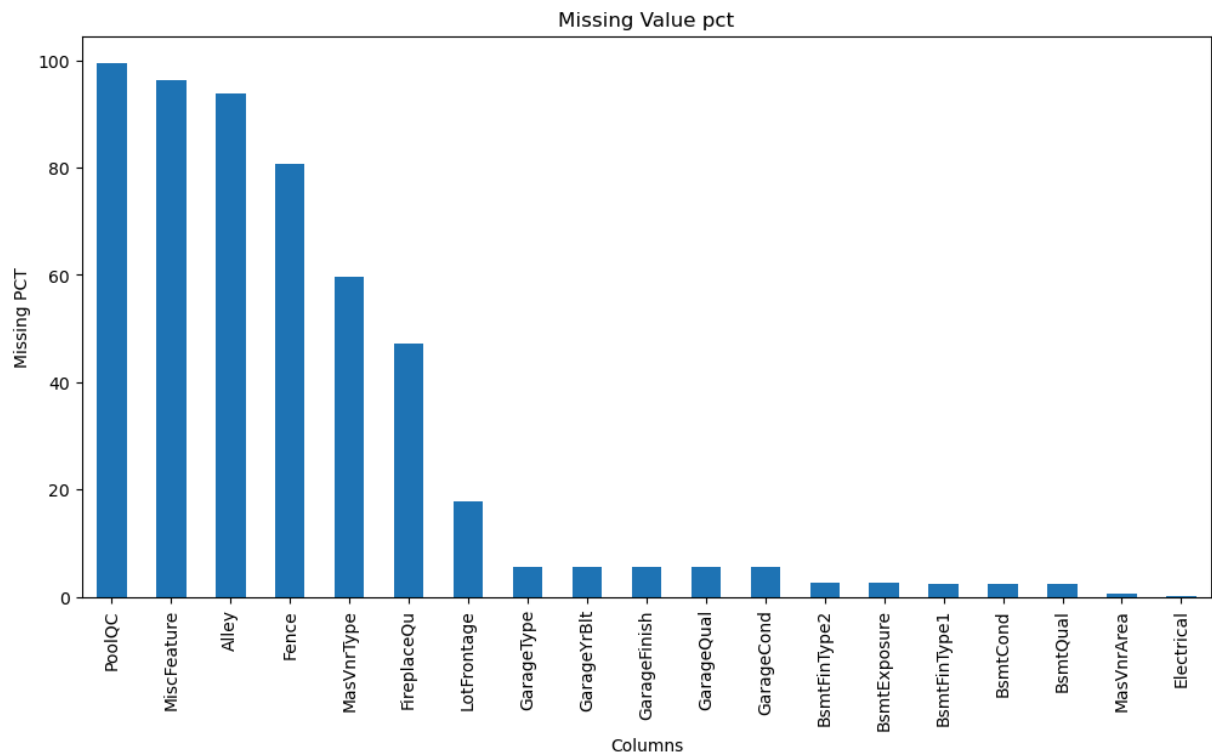
```
In [12]: # Missing Value analysis
missing =df.isnull().sum()
missing=missing[missing>0].sort_values(ascending=False)
missing
```

```
Out[12]: PoolQC          1453
MiscFeature      1406
Alley            1369
Fence            1179
MasVnrType       872
FireplaceQu      690
LotFrontage      259
GarageType        81
GarageYrBlt       81
GarageFinish      81
GarageQual        81
GarageCond        81
BsmtFinType2      38
BsmtExposure      38
BsmtFinType1      37
BsmtCond          37
BsmtQual          37
MasVnrArea        8
Electrical        1
dtype: int64
```

```
In [15]: missing_pct=(df.isnull().sum()/len(df)*100)
missing_pct=missing_pct[missing_pct>0].sort_values(ascending=False)
missing_pct
```

```
Out[15]: PoolQC          99.520548
MiscFeature      96.301370
Alley            93.767123
Fence            80.753425
MasVnrType       59.726027
FireplaceQu      47.260274
LotFrontage      17.739726
GarageType        5.547945
GarageYrBlt       5.547945
GarageFinish      5.547945
GarageQual        5.547945
GarageCond        5.547945
BsmtFinType2      2.602740
BsmtExposure      2.602740
BsmtFinType1      2.534247
BsmtCond          2.534247
BsmtQual          2.534247
MasVnrArea        0.547945
Electrical        0.068493
dtype: float64
```

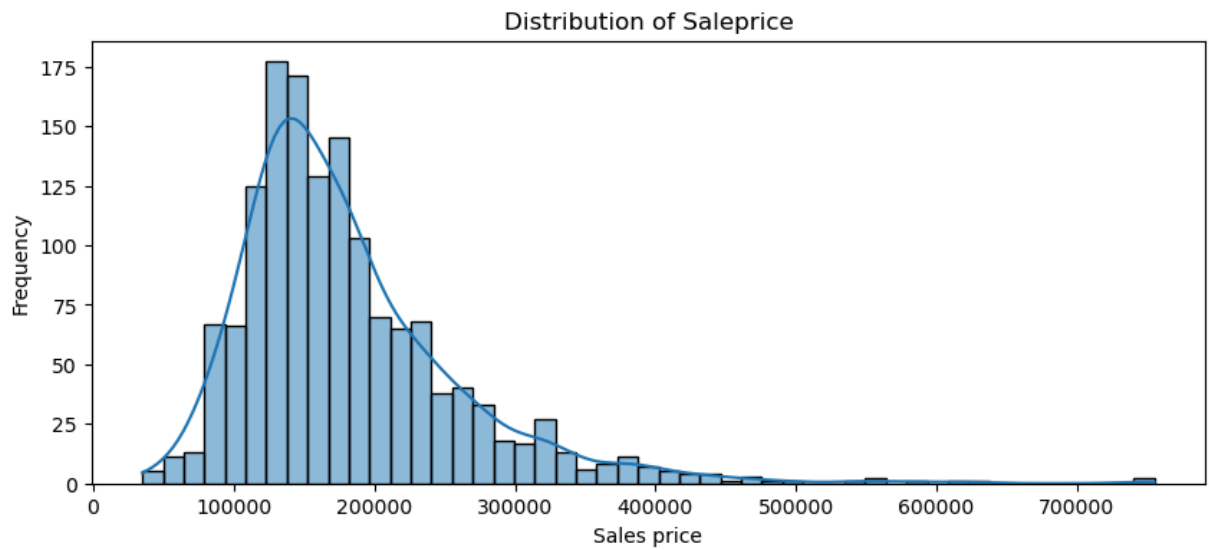
```
In [16]: plt.figure(figsize=(12,6))
missing_pct.plot(kind='bar')
plt.title('Missing Value pct')
plt.xlabel('Columns')
plt.ylabel('Missing PCT')
plt.show()
```



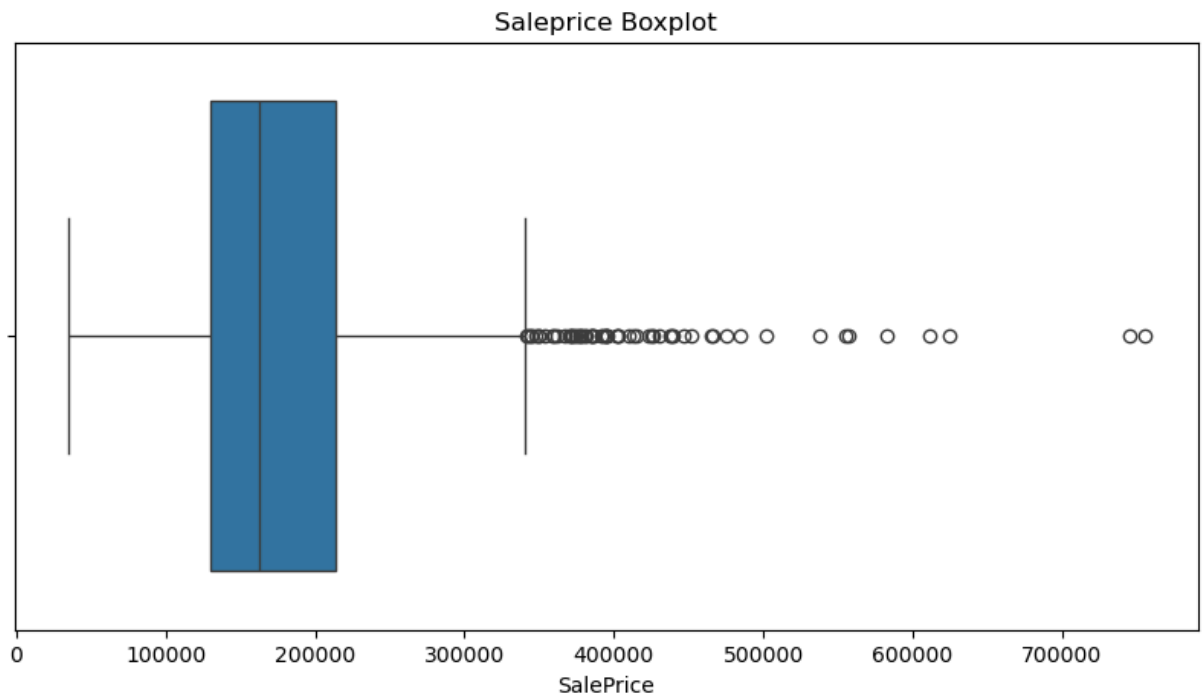
```
In [20]: # Target Variable Analysis
df['SalePrice'].describe()
```

```
Out[20]: count      1460.000000
mean      180921.195890
std       79442.502883
min       34900.000000
25%      129975.000000
50%      163000.000000
75%      214000.000000
max       755000.000000
Name: SalePrice, dtype: float64
```

```
In [21]: plt.figure(figsize=(10,4))
sns.histplot(
    df['SalePrice'],
    kde=True
)
plt.title('Distribution of Saleprice')
plt.xlabel('Sales price')
plt.ylabel('Frequency')
plt.show()
```



```
In [22]: # Boxplot
plt.figure(figsize=(10,5))
sns.boxplot(x=df['SalePrice'])
plt.title('Saleprice Boxplot')
plt.show()
```



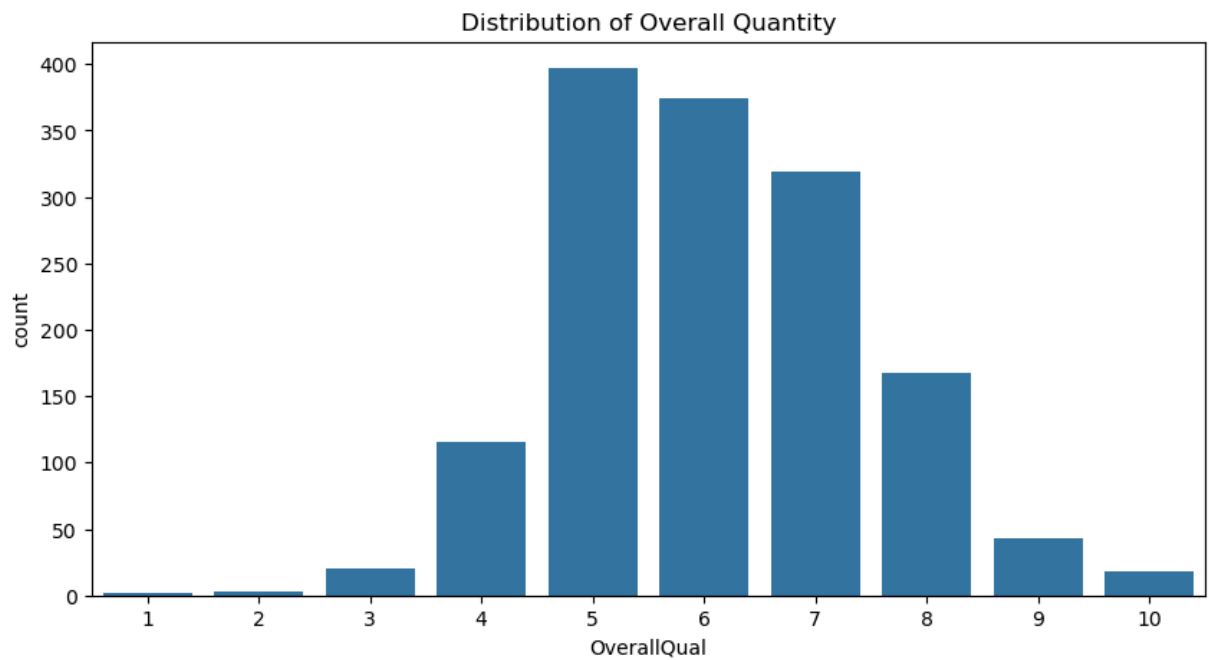
7. EDA

- 7.1 Univariate analysis
- 7.2 Bivariate analysis
- 7.3 Multivariate Analysis

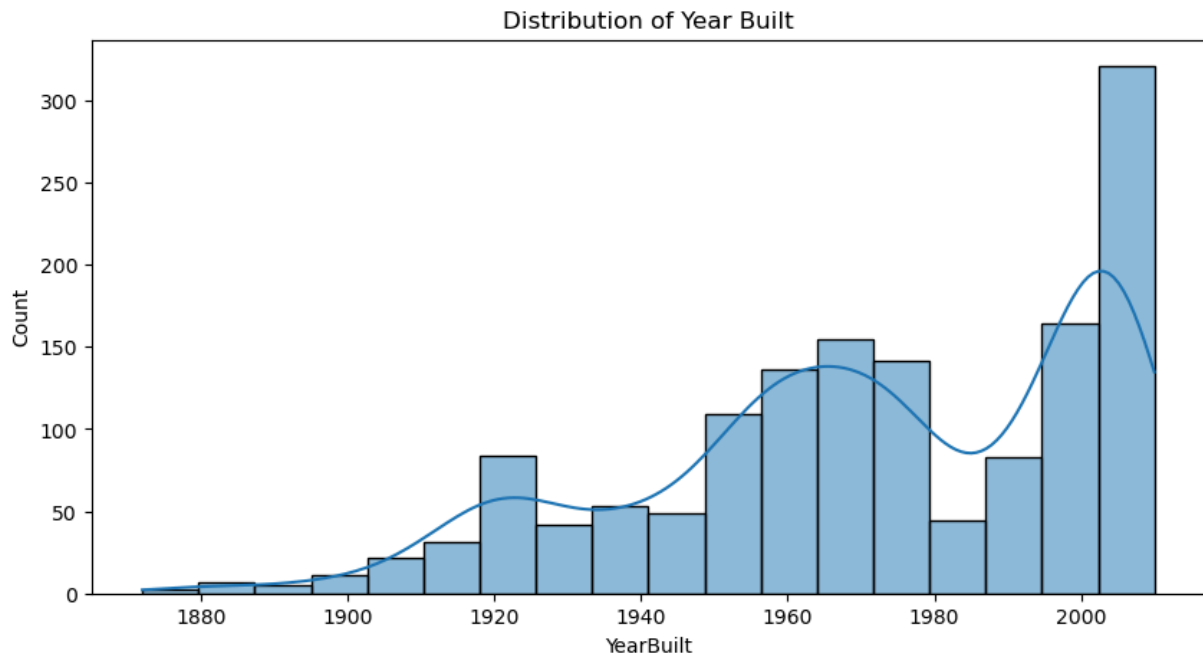
Univariate Analysis :

Univariate Analysis means studying one Variable at a Time

```
In [24]: # Overall Distribution
plt.figure(figsize=(10,5))
sns.countplot(
    data=df,
    x='OverallQual'
)
plt.title('Distribution of Overall Quantity')
plt.show()
```



```
In [25]: # Living area
plt.figure(figsize=(10,5))
sns.histplot(
    df['YearBuilt'],
    kde=True
)
plt.title('Distribution of Year Built')
plt.show()
```



Bivaraiate Analysis :

Bivariate Analysis means studying the relationship between two Variables

```
In [26]: df.columns
```

```
Out[26]: Index(['MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street', 'Alley',
               'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope',
               'Neighborhood', 'Condition1', 'Condition2', 'BldgType', 'HouseStyle',
               'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'RoofStyle',
               'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'MasVnrArea',
               'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond',
               'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1', 'BsmtFinType2',
               'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating', 'HeatingQC',
               'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF',
               'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath',
               'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual', 'TotRmsAbvGrd',
               'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType', 'GarageYrBlt',
               'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual', 'GarageCond',
               'PavedDrive', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch',
               'ScreenPorch', 'PoolArea', 'PoolQC', 'Fence', 'MiscFeature', 'MiscVal',
               'MoSold', 'YrSold', 'SaleType', 'SaleCondition', 'SalePrice'],
              dtype='object')
```

```
In [29]: sns.scatterplot(data=df,x=df['GrLivArea'],y=df['SalePrice'])
plt.title('Living Area vs Sales Price')
plt.show() # Generally as Living area Incraese house price tends to incraese
```



Multivariate Analysis

```
In [30]: selected_features = [  
    "SalePrice",  
    "OverallQual",  
    "GrLivArea",  
    "GarageCars",  
    "TotalBsmntSF",  
    "1stFlrSF",  
    "YearBuilt",  
    "FullBath",  
    "BedroomAbvGr",  
    "TotRmsAbvGrd",  
    "GarageArea"  
]
```

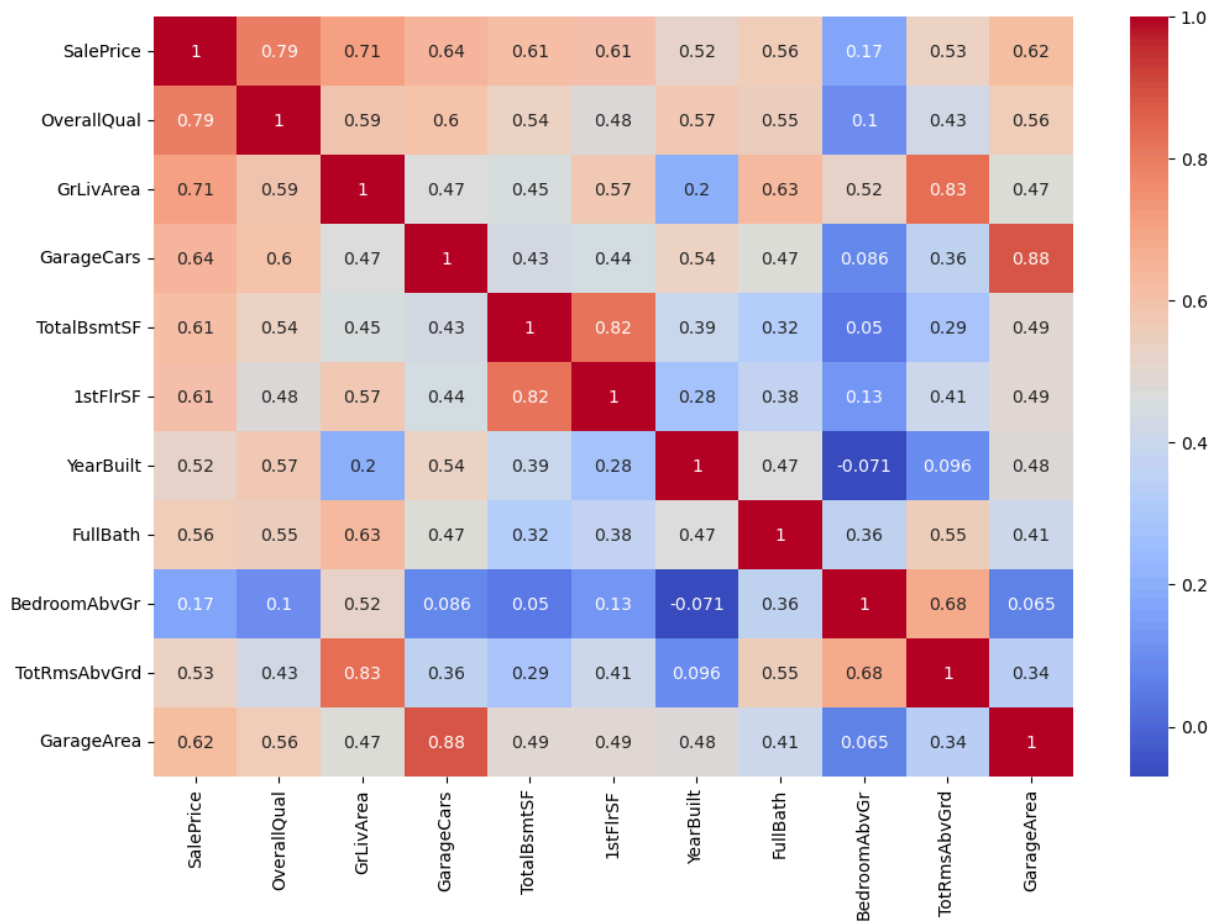
```
In [35]: corr_matrix = df[selected_features].corr()  
corr_matrix
```

Out[35]:

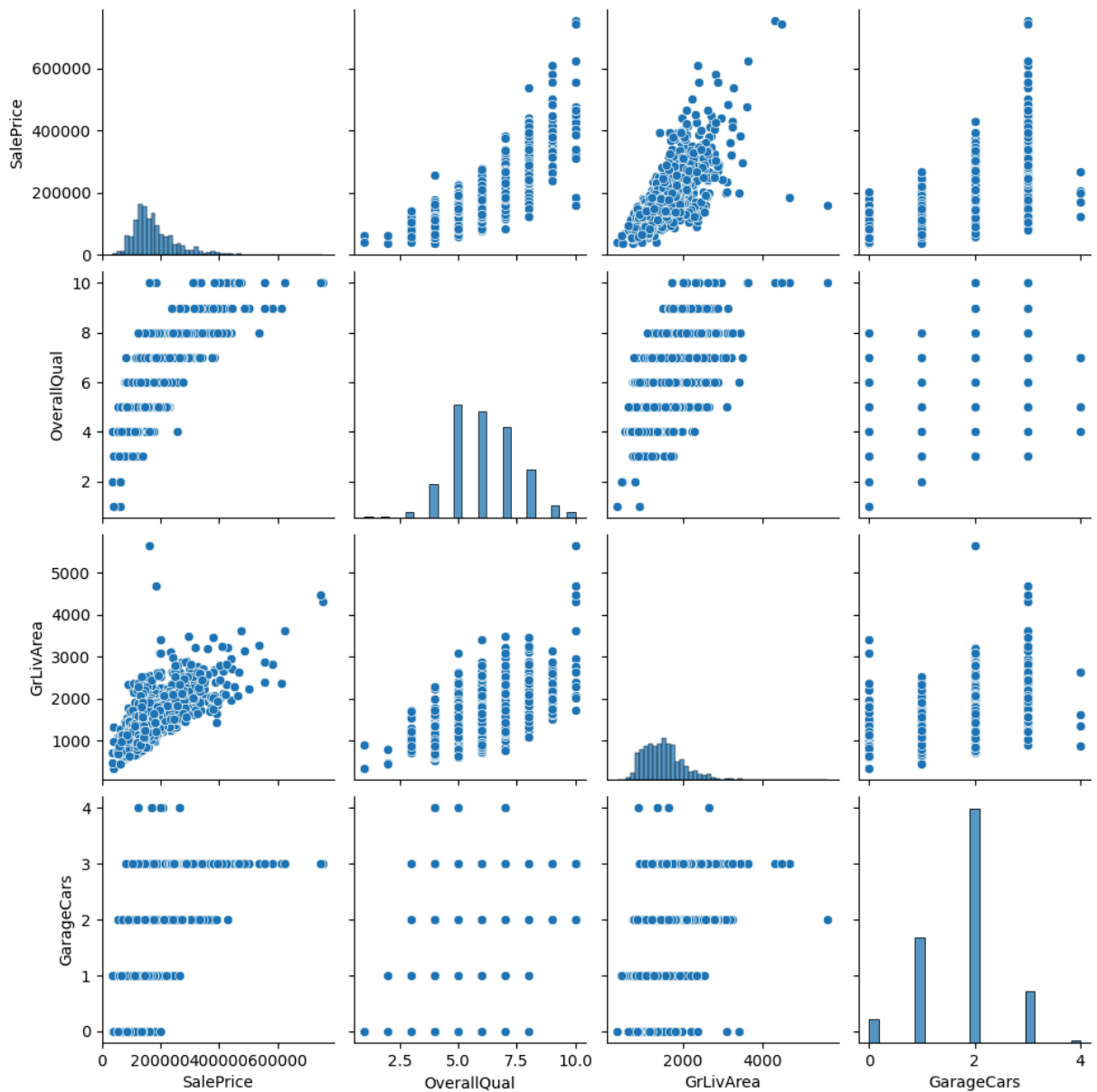
	SalePrice	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	1stFlrSF	Yea
SalePrice	1.000000	0.790982	0.708624	0.640409	0.613581	0.605852	0.5
OverallQual	0.790982	1.000000	0.593007	0.600671	0.537808	0.476224	0.5
GrLivArea	0.708624	0.593007	1.000000	0.467247	0.454868	0.566024	0.1
GarageCars	0.640409	0.600671	0.467247	1.000000	0.434585	0.439317	0.5
TotalBsmtSF	0.613581	0.537808	0.454868	0.434585	1.000000	0.819530	0.3
1stFlrSF	0.605852	0.476224	0.566024	0.439317	0.819530	1.000000	0.2
YearBuilt	0.522897	0.572323	0.199010	0.537850	0.391452	0.281986	1.0
FullBath	0.560664	0.550600	0.630012	0.469672	0.323722	0.380637	0.4
BedroomAbvGr	0.168213	0.101676	0.521270	0.086106	0.050450	0.127401	-0.0
TotRmsAbvGrd	0.533723	0.427452	0.825489	0.362289	0.285573	0.409516	0.0
GarageArea	0.623431	0.562022	0.468997	0.882475	0.486665	0.489782	0.4

```
In [33]: plt.figure(figsize=(12,8))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm'
)
```

Out[33]: <Axes: >



```
In [34]: sns.pairplot(df[
            [
                "SalePrice",
                "OverallQual",
                "GrLivArea",
                "GarageCars"
            ]
        ])
plt.show()
```



```
In [37]: # Identify Highly Correlated Features
corr_with_target= (
    df.corr(numeric_only=True)['SalePrice'].
    sort_values(ascending=False)
)
corr_with_target.head(10)
```

```
Out[37]: SalePrice      1.000000
OverallQual    0.790982
GrLivArea      0.708624
GarageCars     0.640409
GarageArea     0.623431
TotalBsmntSF   0.613581
1stFlrSF       0.605852
FullBath       0.560664
TotRmsAbvGrd   0.533723
YearBuilt      0.522897
Name: SalePrice, dtype: float64
```

In [38]: *# Feature selection*

```
features = [  
    "OverallQual",  
    "GrLivArea",  
    "GarageCars",  
    "TotalBsmtSF",  
    "1stFlrSF",  
    "YearBuilt",  
    "FullBath",  
    "BedroomAbvGr",  
    "TotRmsAbvGrd",  
    "GarageArea"  
]  
x = df[features]  
y = df["SalePrice"]
```

In [39]: *# Missing check*

```
x.isna().sum()
```

Out[39]: OverallQual 0
GrLivArea 0
GarageCars 0
TotalBsmtSF 0
1stFlrSF 0
YearBuilt 0
FullBath 0
BedroomAbvGr 0
TotRmsAbvGrd 0
GarageArea 0
dtype: int64

In [40]: *# Train-test Split :*

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

In [42]: print('x_train = ',x_train.shape)
print('y_train = ',y_train.shape)
print('x_test = ',x_test.shape)
print('y_test = ',y_test.shape)

```
x_train = (1168, 10)  
y_train = (1168,)  
x_test = (292, 10)  
y_test = (292,)
```

In [44]: *# Feature scaling*

```
scaler= StandardScaler()  
x_train=scaler.fit_transform(x_train)  
x_test=scaler.transform(x_test)
```

In [46]: *# Model Build*

```
model=LinearRegression()  
model.fit(x_train,y_train)
```

Out[46]:

LinearRegression
LinearRegression()

In [51]:

```
# Y=Mn*Xn +C
# Model coefficents
coefficeint=pd.DataFrame(
    {'Features':features,
     'Coefficient' : model.coef_
    })
print('Slope/weigth of each input features =\n',coefficeint)
print('intercept/ Bias of model is ',model.intercept_)
```

Slope/weigth of each input features =

	Features	Coefficient
0	OverallQual	27033.061741
1	GrLivArea	26006.506943
2	GarageCars	8202.129179
3	TotalBsmtSF	6957.377144
4	1stFlrSF	4307.086117
5	YearBuilt	11027.679572
6	FullBath	-2270.521017
7	BedroomAbvGr	-6839.246748
8	TotRmsAbvGrd	5400.722065
9	GarageArea	2514.019943

intercept/ Bias of model is 181441.54195205483

In [62]:

```
##### testing evaluation
# Prediction
y_pred=model.predict(
    x_test
)
# Model evaluation
mae=mean_absolute_error(y_test,y_pred)
print(mae)
acc= r2_score(y_test,y_pred)
print(acc)
```

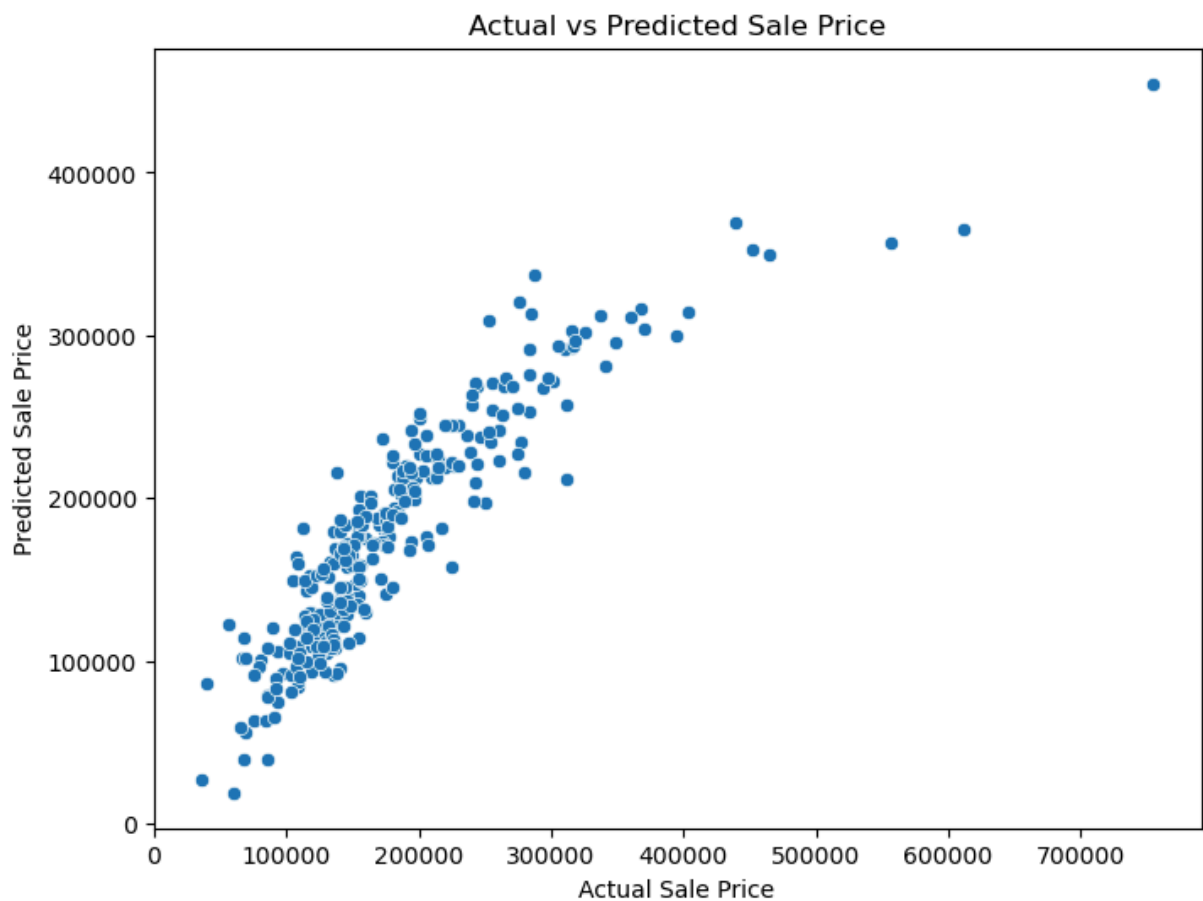
24767.431867529223
0.803382742439527

In [60]:

```
##### Training evaluation
# Prediction
y_pred_train=model.predict(
    x_train
)
# Model evaluation
mae=mean_absolute_error(y_train,y_pred_train)
print(mae)
acc= r2_score(y_train,y_pred_train)
print(acc)
```

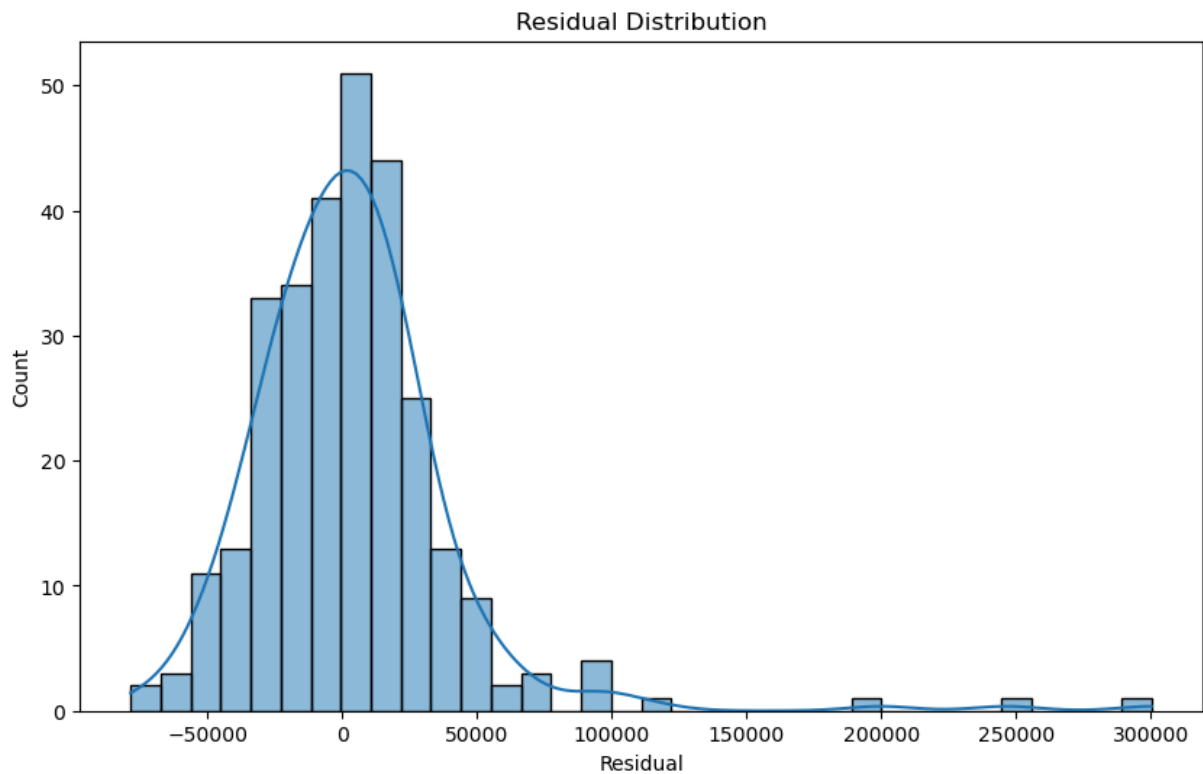
23845.016073262897
0.7643134046743889

```
In [58]: # Actual vs predicted graph
plt.figure(figsize=(8, 6))
sns.scatterplot(
    x=y_test,
    y=y_pred
)
plt.xlabel("Actual Sale Price")
plt.ylabel("Predicted Sale Price")
plt.title("Actual vs Predicted Sale Price")
plt.show()
```



19. Error analysis 20. Save model 21. Predict new house price

```
In [66]: # 19. Error analysis
residual= y_test-y_pred
plt.figure(figsize=(10, 6))
sns.histplot(
    residual,
    kde=True
)
plt.title("Residual Distribution")
plt.xlabel("Residual")
plt.show()
```



In [67]: model

Out[67]: **LinearRegression** ⓘ ?

LinearRegression()

```
In [76]: ##### Deploy This Model
from sklearn.impute import KNNImputer
import gradio as gr
import pandas as pd
def predict_house_price(
    OverallQual,
    GrLivArea,
    GarageCars,
    TotalBsmtSF,
    FirstFlrSF,
    YearBuilt,
    FullBath,
    BedroomAbvGr,
    TotRmsAbvGrd,
    GarageArea,
):
    new_house = pd.DataFrame({
        "OverallQual": [OverallQual],
        "GrLivArea": [GrLivArea],
        "GarageCars": [GarageCars],
        "TotalBsmtSF": [TotalBsmtSF],
        "1stFlrSF": [FirstFlrSF],
        "YearBuilt": [YearBuilt],
        "FullBath": [FullBath],
```

```

    "BedroomAbvGr": [BedroomAbvGr],
    "TotRmsAbvGrd": [TotRmsAbvGrd],
    "GarageArea": [GarageArea],
})

# Scale the new sample using your fitted scaler instance
new_house_scaled = scaler.transform(new_house)

# Prediction
prediction = model.predict(new_house_scaled)
return f"Predicted House Price: ₹ {prediction[0]:,.2f}"

```

```

In [77]: from sklearn.impute import KNNImputer
interface = gr.Interface(
    fn=predict_house_price,
    inputs=[
        gr.Number(
            label="Overall Quality",
            value=7
        ),
        gr.Number(
            label="Living Area (sq ft)",
            value=1800
        ),
        gr.Number(
            label="Garage Cars",
            value=2
        ),
        gr.Number(
            label="Total Basement Area (sq ft)",
            value=1000
        ),
        gr.Number(
            label="First Floor Area (sq ft)",
            value=1000
        ),
        gr.Number(
            label="Year Built",
            value=2005
        ),
        gr.Number(
            label="Full Bathrooms",
            value=2
        ),
        gr.Number(
            label="Bedrooms Above Ground",
            value=3
        ),
        gr.Number(
            label="Total Rooms Above Ground",
            value=7
        )
    ]
)

```

```

    ),
    gr.Number(
        label="Garage Area (sq ft)",
        value=500
    )
],
outputs=gr.Textbox(
    label="Predicted House Price"
),

title="🏠 House Price Prediction",
description=(
    "Linear Regression model for predicting "
    "house sale price."
)
)

```

In [78]: `interface.launch()`

* Running on local URL: `http://127.0.0.1:7863`

* To create a public link, set ``share=True`` in ``launch()``.



House Price Prediction

Linear Regression model for predicting house sale price.

Overall Quality

Living Area (sq ft)

Garage Cars

Total Basement Area (sq ft)

First Floor Area (sq ft)

Out[78]:

In []: